

1. Qual problema o padrão Decorator resolve neste exemplo?

R.: Permite adicionar funcionalidades dinamicamente a um veículo sem modificar a classe VisaoVeiculo.

2. Por que Decorador implementa ComponenteGraficoVeiculo?

R.: Para um objeto decorator continuar sendo tratado como um ComponenteGraficoVeiculo.

3. Qual é a função do atributo componente em Decorador?

R.: Armazenar a referência para o objeto que está sendo usado no decorator.

4. Por que DecoradorModelo e DecoradorMarca herdam de Decorador?

R.: Porque elas estendem o comportamento do objeto decorator.

5. O que aconteceria se a ordem dos decoradores fosse invertida?

R.: A saída seria invertida primeiro a maca depois o modelo.

6. Por que essa solução é mais flexível do que criar subclasses para todas as combinações possíveis?

R.: Porque novas funcionalidades podem ser adicionadas sem criar novas subclasses para cada combinação.

7. Identifique no diagrama os papéis do padrão Decorator.

R.:

Componente	ComponenteGraficoVeiculo
Componente Concreto	VisaoVeiculo
Decorator Abstrato	Decorator
Decorator Concreto	DecoratorModelo
Decorator Concreto	DecoratorMarca

8. Explique por que os decoradores devem delegar a chamada ao objeto decorado antes de adicionar seu comportamento.

R.: Porque o objetivo do Decorator é acrescentar funcionalidades. Ao chamar primeiro componente.exibir(), garante que todo o comportamento anterior seja executado antes da nova funcionalidade ser adicionada.